

FORMAL-AMBIG-RTL: Formally Certified Ambiguity Handling in Large Language Model-Based FSM and RTL Generation

Naveen Kumar

Department of Electronics and Communication Engineering
National Institute of Technology Warangal
Warangal, Telangana, India

Abstract—Large Language Models (LLMs) are increasingly being investigated for natural-language-to-register-transfer-level (RTL) hardware generation. Existing benchmarks have demonstrated that LLMs can translate natural-language descriptions into synthesizable RTL and finite-state machines (FSMs). However, most existing evaluations implicitly assume that a natural-language hardware specification has one unique intended interpretation. This assumption is problematic when a specification is underspecified, because multiple reasonable interpretations can result in behaviorally different hardware.

In parallel, recent research in software engineering has investigated whether LLM-based systems can detect underspecified requirements and ask clarification questions before generating software. However, a hardware-specific methodology that combines ambiguity handling, formal behavioral certification, and LLM-based RTL generation remains insufficiently explored.

This paper proposes *FORMAL-AMBIG-RTL*, a benchmark and evaluation framework for studying ambiguity handling in LLM-based FSM and RTL generation. The proposed methodology constructs candidate ambiguous hardware specifications, derives multiple human-plausible interpretations, generates corresponding reference RTL implementations, and applies formal verification to establish behavioral non-equivalence between the interpretations. A specification is considered *formally certified ambiguous* only when its alternative interpretations are both judged plausible and are formally shown to exhibit distinguishable hardware behavior. LLMs are subsequently evaluated under single-shot and interactive conditions and categorized according to whether they appropriately request clarification, silently assume an interpretation, or produce RTL inconsistent with the certified interpretations.

The framework introduces metrics for ambiguity detection, clarification behavior, interpretation accuracy, misinterpretation, formal correctness, and interaction cost. An initial benchmark is designed around FSM-specific ambiguity categories including reset semantics, transition priority, output semantics, timing behavior, and state initialization. The proposed framework aims to provide a reproducible methodology for evaluating whether LLM-based hardware generation systems can reason about uncertainty in hardware requirements rather than merely generate syntactically valid RTL.

Index Terms—Large Language Models, RTL generation, FSM, formal verification, hardware design automation, ambiguity detection, clarification, electronic design automation, LLM benchmarks.

I. INTRODUCTION

Large Language Models (LLMs) have rapidly emerged as potential assistants for electronic design automation (EDA), in-

cluding natural-language-to-register-transfer-level (RTL) generation, hardware verification, assertion generation, and design debugging. Recent studies have demonstrated that LLMs can generate synthesizable Verilog and SystemVerilog from natural-language descriptions and have introduced benchmark datasets for evaluating this capability [1], [2].

Finite-State Machines (FSMs) represent an important class of hardware designs for evaluating natural-language-to-RTL generation. FSM behavior depends directly on state definitions, transition conditions, output semantics, reset behavior, and timing relationships. Recent work such as LLM-FSM has specifically investigated the ability of LLMs to generate FSM-oriented RTL at increasing levels of complexity [3].

However, a fundamental issue remains underexplored in many existing evaluations: *what happens when the natural-language specification itself does not uniquely determine the hardware behavior?*

Consider the following requirement:

“When reset is asserted, the controller returns to the IDLE state.”

This requirement may not explicitly specify whether reset is synchronous or asynchronous. Two hardware designers may therefore produce different implementations while both considering their implementation consistent with the stated requirement.

Consequently, there is an important distinction between:

- 1) an implementation that is incorrect with respect to a clear requirement, and
- 2) an implementation that represents one reasonable interpretation of an underspecified requirement.

Traditional benchmark evaluation against a single reference RTL can fail to distinguish these cases.

At the same time, simply labeling a specification as “ambiguous” is not sufficient for rigorous evaluation. Two candidate interpretations may be behaviorally different while only one of them is linguistically reasonable. Therefore, ambiguity evaluation requires both a semantic and a behavioral criterion.

This paper proposes a framework based on the following principle:

$$\boxed{\text{Certified Ambiguity} = \text{Plausibility} \wedge \text{NonEquivalence}} \quad (1)$$

where plausibility represents independent human or expert agreement that two interpretations are reasonable readings of the specification, and non-equivalence represents formally established behavioral difference between the corresponding RTL implementations.

The resulting benchmark can then be used to investigate whether an LLM:

- recognizes the ambiguity,
- asks an appropriate clarification question,
- silently selects one interpretation, or
- generates behavior inconsistent with all certified interpretations.

The main contribution of this work is therefore not a new formal equivalence algorithm or a general-purpose LLM, but a methodology for using formal verification as an objective behavioral component in evaluating ambiguity handling during LLM-based hardware generation.

The remainder of this paper is organized as follows. Section II discusses related work. Section III defines the research gap and problem formulation. Section IV presents the proposed framework. Section V describes the ambiguity taxonomy and benchmark construction. Section VI presents the experimental methodology. Section VII defines the evaluation metrics. Section VIII discusses expected analysis and hypotheses. Section IX presents threats to validity. Section X concludes the paper.

II. RELATED WORK

A. LLM-Based RTL Generation

RTL2LLM introduced an open-source benchmark for evaluating RTL generation from natural-language hardware descriptions [1]. The benchmark established the importance of evaluating generated RTL beyond natural-language quality, including syntax and functional correctness.

Subsequent work has expanded the scale of RTL-generation datasets and benchmarks. OpenLLM-RTL provides additional datasets and evaluation resources for LLM-assisted RTL generation and verification [2].

These studies demonstrate the feasibility of using LLMs for hardware generation. However, their primary evaluation objective is generally whether generated RTL matches an expected implementation or satisfies a predefined functional specification.

The proposed work addresses a different question: whether the LLM can recognize that the specification itself permits multiple reasonable hardware interpretations.

B. LLM-Based FSM Generation

FSMs provide a structured representation of sequential hardware and are particularly suitable for evaluating LLM reasoning.

LLM-FSM introduced a scalable benchmark for finite-state reasoning in RTL generation, including automatically generated FSM problems and verified RTL/testbench pairs [3]. Such work demonstrates that FSM complexity can significantly affect LLM generation performance.

The proposed framework complements this direction by introducing controlled specification ambiguity rather than focusing only on complexity.

Specifically, existing FSM generation can be represented as:

$$S \rightarrow R^* \quad (2)$$

where S is a natural-language specification and R^* is the intended RTL implementation.

FORMAL-AMBIG-RTL instead considers:

$$S \rightarrow \{I_A, I_B, \dots\} \rightarrow \{R_A, R_B, \dots\} \quad (3)$$

where multiple interpretations may be consistent with the natural language.

C. Ambiguity Handling in LLMs

Recent software-engineering research has begun treating underspecification as an explicit LLM evaluation problem. Ambig-SWE investigates interactive agents that detect underspecified software tasks, ask clarification questions, and incorporate the resulting information into their final solutions [4].

This research establishes the importance of ambiguity-aware LLM interaction. However, hardware specifications have additional behavioral dimensions involving clocking, reset, sequential state, transition timing, and output semantics.

The proposed work transfers the ambiguity-handling question into the hardware-design domain while adding formal behavioral certification.

D. Formal Verification and LLM-Generated Hardware

Formal verification has long been used to establish equivalence and correctness of hardware designs. More recently, researchers have investigated combining LLM generation with formal verification.

Proof2Silicon, for example, investigates formally verified code and hardware generation and demonstrates the potential of using formal methods as part of LLM-driven hardware workflows [5].

The proposed work uses formal verification for a different purpose. Instead of using formal verification only to verify the final LLM output, it uses formal verification during *benchmark construction* to determine whether alternative interpretations are behaviorally distinguishable.

III. RESEARCH GAP AND PROBLEM FORMULATION

Existing research can broadly be divided into three streams:

- 1) natural-language-to-RTL generation,
- 2) ambiguity and clarification handling in LLMs, and
- 3) formal verification of hardware.

These streams have largely developed independently.

TABLE I
POSITIONING OF THE PROPOSED RESEARCH

Research Direction	Typical Objective
LLM $\rightarrow$ RTL	Generate functionally correct RTL from a specified requirement.
FSM generation	Recover or generate FSM behavior with increasing complexity.
LLM ambiguity handling	Detect underspecification and request clarification.
Formal verification	Prove equivalence or correctness of hardware implementations.
Proposed work	Combine hardware ambiguity, formal behavioral certification, and LLM clarification evaluation.

Table I summarizes the distinction.

The central research gap can therefore be stated as follows:

Existing RTL-generation benchmarks predominantly evaluate LLM outputs against a predetermined intended behavior, while existing ambiguity-handling benchmarks primarily target software tasks. A hardware-specific evaluation methodology is needed in which alternative plausible interpretations of a natural-language specification are formally certified as behaviorally distinct before evaluating LLM ambiguity-handling behavior.

This leads to the central research question:

Can formal verification provide objective behavioral ground truth for evaluating ambiguity handling in LLM-based RTL generation?

IV. PROBLEM DEFINITION

Let:

$$S \quad (4)$$

represent a natural-language hardware specification. Suppose two interpretations are generated:

$$I_A(S), \quad I_B(S). \quad (5)$$

The corresponding RTL implementations are:

$$R_A = f(I_A) \quad (6)$$

and

$$R_B = f(I_B). \quad (7)$$

Formal verification determines whether:

$$R_A \equiv R_B \quad (8)$$

or:

$$R_A \not\equiv R_B. \quad (9)$$

However, non-equivalence alone does not establish ambiguity.

Therefore, let:

$$P(I_A, I_B | S) \quad (10)$$

represent the plausibility that both interpretations are reasonable readings of S .

The proposed certified ambiguity condition is:

$$CA(S) = P(I_A, I_B | S) \wedge (R_A \not\equiv R_B). \quad (11)$$

Only cases satisfying this condition are included as certified ambiguity instances.

V. FORMAL-AMBIG-RTL FRAMEWORK

The proposed framework consists of six major stages:

- 1) ambiguity candidate generation,
- 2) alternative interpretation construction,
- 3) RTL generation for each interpretation,
- 4) formal non-equivalence checking,
- 5) human plausibility validation, and
- 6) LLM ambiguity-handling evaluation.

A. Overall Architecture

Fig. 1 illustrates the proposed architecture.

VI. HARDWARE AMBIGUITY TAXONOMY

The initial benchmark focuses on five classes of ambiguity that are particularly relevant to FSM and sequential RTL generation.

A. Reset Semantics

Reset ambiguity includes:

- synchronous versus asynchronous reset,
- active-high versus active-low reset,
- reset priority versus enable priority, and
- reset behavior during simultaneous control events.

For example:

“When reset is asserted, the FSM returns to IDLE.” may leave the timing semantics of reset unspecified.

B. Output Semantics

Output ambiguity includes:

- Mealy versus Moore behavior,
- registered versus combinational outputs,
- current-state versus next-state output semantics.

C. Transition Semantics

Transition ambiguity includes:

- priority versus parallel transitions,
- overlapping versus non-overlapping sequence detection,
- simultaneous input conditions, and
- unspecified default transitions.

TABLE II
PROPOSED INITIAL BENCHMARK DISTRIBUTION

Ambiguity Category	Target Cases
Reset semantics	30
Output semantics	30
Transition semantics	40
Timing semantics	30
State semantics	30
Total	160

D. Timing Semantics

Timing ambiguity includes:

- same-cycle versus next-cycle responses,
- input sampling behavior,
- state transition timing, and
- output update timing.

E. State Semantics

State-related ambiguity includes:

- unspecified initial state,
- illegal-state recovery,
- default-state behavior, and
- recovery from unreachable states.

VII. BENCHMARK CONSTRUCTION

A major objective of FORMAL-AMBIG-RTL is to construct a benchmark that separates genuine ambiguity from simple implementation error.

A. Benchmark Structure

Each benchmark instance contains:

- 1) a natural-language specification,
- 2) ambiguity category,
- 3) interpretation A,
- 4) interpretation B,
- 5) RTL implementation A,
- 6) RTL implementation B,
- 7) formal verification result,
- 8) counterexample trace where available,
- 9) plausibility annotations, and
- 10) ambiguity-severity information.

A proposed initial benchmark size is approximately 160 cases.

The exact final number of certified cases is intentionally left as an experimental outcome because candidate generation may produce cases that fail either the plausibility or formal non-equivalence criterion.

B. Control Cases

The benchmark should also contain unambiguous control cases.

Three major groups are proposed:

- 1) **Unambiguous cases:** the requirement clearly defines one behavior.

TABLE III
ILLUSTRATIVE BEHAVIORAL COUNTEREXAMPLE

Cycle	Reset	Input	State-A	State-B
0	0	0	IDLE	IDLE
1	1	0	IDLE	ERROR
2	0	0	IDLE	ERROR

- 2) **Certified ambiguous cases:** multiple plausible interpretations are formally non-equivalent.
- 3) **Negative ambiguity candidates:** candidate pairs that fail the plausibility or non-equivalence test.

Control cases are important because an LLM that asks clarification questions for every specification should not automatically be considered successful.

VIII. FORMAL VERIFICATION METHODOLOGY

Formal verification is used at two different points in the proposed framework.

First, it is used during benchmark construction to establish whether two interpretations produce distinguishable behavior.

Second, it is used after LLM generation to determine whether the generated RTL corresponds to one of the certified interpretations.

A. Reference Equivalence

For two reference implementations:

$$R_A, R_B$$

the verification objective is:

$$R_A \equiv R_B \quad (12)$$

or:

$$R_A \not\equiv R_B. \quad (13)$$

A counterexample trace can be retained when the designs are shown to be non-equivalent.

For example:

The exact counterexample depends on the ambiguity category and the formal verification problem.

B. LLM RTL Classification

For generated RTL R_{LLM} , formal equivalence is evaluated against both certified interpretations:

$$R_{LLM} \equiv R_A \quad (14)$$

and:

$$R_{LLM} \equiv R_B. \quad (15)$$

The resulting classification is:

$$Class(R_{LLM}) = \begin{cases} A, & R_{LLM} \equiv R_A \\ B, & R_{LLM} \equiv R_B \\ AB, & R_{LLM} \equiv R_A \wedge R_{LLM} \equiv R_B \\ M, & \text{otherwise.} \end{cases} \quad (16)$$

Because certified ambiguity requires $R_A \not\equiv R_B$, the *AB* category should generally occur only in exceptional situations such as insufficiently discriminating verification assumptions.

The important failure category is:

$M = \text{Misinterpretation.}$

IX. HUMAN PLAUSIBILITY VALIDATION

Formal non-equivalence does not prove that two interpretations are equally reasonable readings of the natural-language specification.

Therefore, independent human plausibility validation is required.

For each candidate:

S, I_A, I_B

annotators will answer whether:

- 1) interpretation A is reasonable,
- 2) interpretation B is reasonable, and
- 3) the original specification is sufficiently ambiguous to permit both interpretations.

A candidate is accepted as certified only when predefined agreement criteria are satisfied.

For example, with n annotators:

$$P_A = \frac{N_A^{\text{reasonable}}}{n} \quad (17)$$

and:

$$P_B = \frac{N_B^{\text{reasonable}}}{n}. \quad (18)$$

A threshold τ may be defined before experimentation:

$$P_A \geq \tau \wedge P_B \geq \tau. \quad (19)$$

Inter-annotator agreement should also be reported using an appropriate statistic such as Cohen's kappa for two annotators or Fleiss' kappa for multiple annotators.

X. LLM EXPERIMENTAL DESIGN

The proposed evaluation compares single-shot and interactive generation.

A. Single-Shot Condition

The model receives only the natural-language specification:

$$S \rightarrow LLM \rightarrow R_{LLM}. \quad (20)$$

The model is not informed that the specification has been labeled ambiguous.

B. Interactive Condition

The model receives the same specification but is permitted to request clarification:

$$S \rightarrow LLM \rightarrow Q \rightarrow Answer \rightarrow LLM \rightarrow R_{LLM}. \quad (21)$$

The clarification answer should be generated from the certified interpretation information using a controlled experimental protocol.

C. Behavioral Categories

The model output is classified into three principal categories.

1) *ASK*: The LLM identifies the unresolved behavioral dimension and asks a clarification question.

Example:

"Should RESET be synchronous or asynchronous?"

2) *ASSUME*: The LLM selects an interpretation without asking for clarification.

Example:

"The design will use a synchronous reset."

3) *MISINTERPRET*: The generated RTL is formally inconsistent with both certified interpretations.

This distinction is important because silent assumption and genuine misinterpretation represent different types of behavior.

XI. RESEARCH QUESTIONS AND HYPOTHESES

The proposed study investigates the following research questions.

A. RQ1

How accurately do LLMs detect formally certified ambiguities in natural-language FSM specifications?

B. RQ2

How does ambiguity type influence whether an LLM asks for clarification or silently assumes an interpretation?

C. RQ3

Does interactive clarification improve the formal correctness of generated RTL compared with single-shot generation?

D. RQ4

How frequently do LLMs generate implementations that correspond to neither certified interpretation?

E. RQ5

Does behavioral ambiguity severity influence clarification behavior and RTL correctness?

F. Hypotheses

The following hypotheses are proposed for experimental testing.

- **H1:** LLMs will not detect all certified hardware ambiguities.
- **H2:** Interactive clarification will improve interpretation accuracy compared with single-shot generation.
- **H3:** LLM behavior will vary across ambiguity categories.
- **H4:** Greater behavioral ambiguity severity will influence clarification behavior.
- **H5:** A measurable fraction of generated RTL will correspond to neither certified interpretation.

These hypotheses are proposed for experimental evaluation and are not presented as established results.

XII. EVALUATION METRICS

A. Ambiguity Detection Rate

$$ADR = \frac{N_{detected}}{N_{certified}} \quad (22)$$

where $N_{detected}$ is the number of certified ambiguity cases in which the LLM appropriately identifies unresolved behavior.

B. Clarification Rate

$$CR = \frac{N_{ASK}}{N_{certified}}. \quad (23)$$

C. Interpretation Accuracy

$$IA = \frac{N_A + N_B}{N_{certified}}. \quad (24)$$

Here, N_A and N_B represent outputs formally matching interpretations A and B, respectively.

D. Misinterpretation Rate

$$MIR = \frac{N_M}{N_{certified}}. \quad (25)$$

E. Formal Correctness Rate

$$FCR = \frac{N_{formally\ correct}}{N_{generated}}. \quad (26)$$

F. Clarification Utility

The benefit of interactive clarification can be represented as:

$$CU = \frac{FCR_{interactive} - FCR_{single}}{C_{interaction}}. \quad (27)$$

Interaction cost may incorporate token usage, number of turns, and latency:

$$C_{interaction} = \alpha T + \beta U + \gamma L \quad (28)$$

where:

Algorithm 1 FORMAL-AMBIG-RTL Benchmark Construction

Require: Candidate specification set $\mathcal{S}$

Ensure: Certified ambiguity benchmark $\mathcal{B}$

```

1:  $\mathcal{B} \leftarrow \emptyset$ 
2: for all  $S \in \mathcal{S}$  do
3:   Generate candidate interpretations  $I_A, I_B$ 
4:   Generate reference RTL  $R_A, R_B$ 
5:   Perform formal comparison
6:   if  $R_A \neq R_B$  then
7:     Obtain independent plausibility annotations
8:     if Both interpretations satisfy plausibility threshold
       then
9:       Add  $(S, I_A, I_B, R_A, R_B)$  to  $\mathcal{B}$ 
10:    end if
11:  end if
12: end for
13: return  $\mathcal{B}$ 

```

- T is the number of additional interaction turns,
- U is additional token usage,
- L is additional latency,
- α, β, γ are predefined normalization coefficients.

XIII. BEHAVIORAL AMBIGUITY SEVERITY

Not all formally non-equivalent interpretations have identical practical consequences.

Two interpretations may differ only under a rare input sequence, whereas another pair may generate substantially different state trajectories.

Therefore, the proposed study introduces the concept of *Behavioral Ambiguity Severity* (BAS).

Let:

$$BAS = D(R_A, R_B) \quad (29)$$

where D represents an experimentally defined measure of behavioral difference.

Depending on the size of the FSM, D may be estimated using:

- exhaustive state-space exploration,
- bounded model checking,
- counterexample frequency,
- state-transition divergence, or
- controlled input-sequence analysis.

The objective is not to claim a universal definition of ambiguity severity, but to investigate whether the behavioral consequences of ambiguity influence LLM clarification behavior.

XIV. EXPERIMENTAL PROTOCOL

The planned experimental workflow is shown below.

The final experimental workflow is:

- 1) construct candidate specifications;
- 2) generate alternative interpretations;

- 3) generate verified reference RTL;
- 4) formally test behavioral equivalence;
- 5) conduct plausibility validation;
- 6) retain certified ambiguity cases;
- 7) evaluate selected LLMs;
- 8) formally analyze generated RTL;
- 9) classify ASK, ASSUME, and MISINTERPRET behavior;
- 10) compare single-shot and interactive performance.

XV. EXPECTED RESULTS AND ANALYSIS PLAN

No experimental results are claimed in this paper prior to execution of the proposed benchmark.

The final experimental study is expected to produce three major analysis groups.

A. Analysis A: Ambiguity Detection

The first analysis will determine:

$$ADR(Model, Category) \quad (30)$$

across models and ambiguity categories.

This will reveal whether models detect some hardware ambiguities more reliably than others.

B. Analysis B: Clarification Versus Assumption

The second analysis will compare:

$$ASK \text{ versus } ASSUME.$$

The objective is not to define assumption as inherently incorrect. Instead, the study will examine whether silent assumptions are consistent with the certified interpretation set.

C. Analysis C: Interactive Benefit

The third analysis will compare:

$$FCR_{single} \quad (31)$$

against:

$$FCR_{interactive}. \quad (32)$$

The final paper will report the experimentally measured difference.

XVI. RESULTS REPORTING TEMPLATES

The following tables are intentionally left as placeholders for the experimental stage.

XVII. THREATS TO VALIDITY

A. Human Annotation Bias

Determining whether two interpretations are reasonable involves human judgment. Different experts may disagree.

To reduce this threat, multiple independent annotators should be used and inter-annotator agreement should be reported.

TABLE IV
BENCHMARK CERTIFICATION RESULTS

Category	Candidates	Non-Equiv.	Certified
Reset	TBD	TBD	TBD
Output	TBD	TBD	TBD
Transition	TBD	TBD	TBD
Timing	TBD	TBD	TBD
State	TBD	TBD	TBD
Total	TBD	TBD	TBD

TABLE V
LLM AMBIGUITY-HANDLING RESULTS

Model	ADR	CR	IA	MIR	FCR
Model A	TBD	TBD	TBD	TBD	TBD
Model B	TBD	TBD	TBD	TBD	TBD
Model C	TBD	TBD	TBD	TBD	TBD
Model D	TBD	TBD	TBD	TBD	TBD

B. Formal Verification Assumptions

Formal equivalence results depend on the assumptions, interfaces, clocking models, reset models, and verification environment.

Therefore, the final implementation should report the exact formal verification assumptions.

C. Benchmark Contamination

Publicly available RTL benchmarks may have appeared in training data for some LLMs. This can affect measured performance.

Where practical, benchmark instances should be procedurally generated, held out, or generated after the evaluated models' training cutoff.

D. Model Dependence

Results obtained from a limited set of models may not generalize to all LLMs.

The final evaluation should therefore include models with different architectures, sizes, and hardware-generation capabilities where resources permit.

E. FSM Scope

The initial benchmark focuses on FSM and sequential RTL behavior. Therefore, conclusions may not immediately generalize to datapaths, memories, processors, or complete SoC-level specifications.

XVIII. REPRODUCIBILITY PLAN

The proposed research should release, subject to licensing and model constraints:

- 1) benchmark specifications;
- 2) ambiguity annotations;
- 3) reference RTL;
- 4) formal verification scripts;
- 5) counterexample traces;
- 6) evaluation prompts;

TABLE VI
SINGLE-SHOT VERSUS INTERACTIVE EVALUATION

Metric	Single-Shot	Interactive	Difference
Interpretation Accuracy	TBD	TBD	TBD
Formal Correctness	TBD	TBD	TBD
Misinterpretation	TBD	TBD	TBD
Tokens	TBD	TBD	TBD
Latency	TBD	TBD	TBD

- 7) generated LLM outputs;
- 8) scoring scripts; and
- 9) statistical-analysis scripts.

A reproducible repository structure may be organized as:

```

FORMAL-AMBIG-RTL/
|
+-- benchmark/
|   +-- specifications/
|   +-- interpretations/
|   +-- rtl/
|   +-- annotations/
|
+-- formal/
|   +-- equivalence/
|   +-- counterexamples/
|
+-- prompts/
|
+-- models/
|
+-- evaluation/
|   +-- classification/
|   +-- metrics/
|   +-- statistics/
|
+-- results/
|
+-- paper/

```

XIX. DISCUSSION

The proposed framework changes the evaluation target for natural-language-to-RTL systems.

Traditional evaluation can be summarized as:

$$\text{Specification} \rightarrow \text{Expected RTL} \rightarrow \text{LLM RTL} \rightarrow \text{Correct/Incorrect} \quad (33)$$

This formulation implicitly assumes that the specification determines one implementation.

FORMAL-AMBIG-RTL instead models:

$$\text{Specification} \rightarrow \{\text{Plausible Interpretation A, Plausible Interpretation B}\} \quad (34)$$

The LLM is therefore evaluated not only for implementation accuracy but also for its behavior under uncertainty.

This distinction is important in practical engineering environments. Human hardware requirements are frequently communicated using informal natural language. Requirements may omit timing information, reset semantics, transition priority, or output-update behavior.

A useful LLM-based hardware assistant should therefore ideally distinguish between:

- 1) requirements that are sufficiently specified,
- 2) requirements that contain unresolved choices, and
- 3) requirements that are internally inconsistent.

The present work focuses specifically on the second category.

XX. CONCLUSION

This paper proposed FORMAL-AMBIG-RTL, a methodology for evaluating ambiguity handling in LLM-based FSM and RTL generation.

The central idea is to combine two independent forms of evidence:

$$\text{Certified Ambiguity} = \text{Human Plausibility} \wedge \text{Formal NonEquivalence} \quad (35)$$

This avoids treating formal non-equivalence alone as proof of linguistic ambiguity.

The proposed framework constructs alternative interpretations of natural-language hardware requirements, generates corresponding reference RTL implementations, formally verifies their behavioral distinction, and uses the resulting certified cases to evaluate LLM behavior.

Three principal LLM behaviors are evaluated:

- 1) appropriate clarification,
- 2) silent assumption, and
- 3) misinterpretation.

The framework additionally proposes metrics for ambiguity detection, interpretation accuracy, formal correctness, clarification utility, and interaction cost.

The primary research objective is therefore not simply to determine whether an LLM can generate valid RTL. Instead, the objective is to determine whether the model can recognize when the specification does not uniquely determine the hardware behavior and appropriately manage that uncertainty.

The proposed methodology provides a foundation for a hardware-specific benchmark at the intersection of LLM-based design automation, requirements ambiguity, and formal verification. Experimental execution of the benchmark is required before quantitative claims can be made regarding the performance of individual LLMs.

REFERENCES

- [1] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model," in *Proc. 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 722–727.
- [2] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation," 2025.

- [3] Y. Wu, B. Gokmen, Z. Xie, P. Li, C. Trippel, P. Raina, and T. Tambe, "LLM-FSM: Scaling Large Language Models for Finite-State Reasoning in RTL Code Generation," 2026.
- [4] S. Vijayvargiya, X. Zhou, A. Yerukola, M. Sap, and G. Neubig, "Ambig-SWE: Interactive Agents to Overcome Underspecificity in Software Engineering," in *Proc. International Conference on Learning Representations (ICLR)*, 2026.
- [5] M. Jha, J. Wan, and D. Chen, "Proof2Silicon: Prompt Repair for Verified Code and Hardware Generation via Reinforcement Learning," 2025.
- [6] A. Cimatti, C. Gario, and S. Tonetta, "Validation Obligations: A Novel Approach to Check Compliance Between Requirements and Their Formal Specification," in *Proc. IEEE/ACM International Conference on Software Engineering*, 2021.
- [7] A. Kuehlmann and F. Krohm, "Equivalence Checking Using Cuts and Heaps," in *Proc. Design Automation Conference*, 1997.
- [8] C. Wolf, "Yosys Open SYNthesis Suite," <https://yosyshq.net/yosys/>.
- [9] C. Wolf, "SymbiYosys: Front-end for Formal Verification Flows," <https://symbiyosys.readthedocs.io/>.
- [10] IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std. 1800.

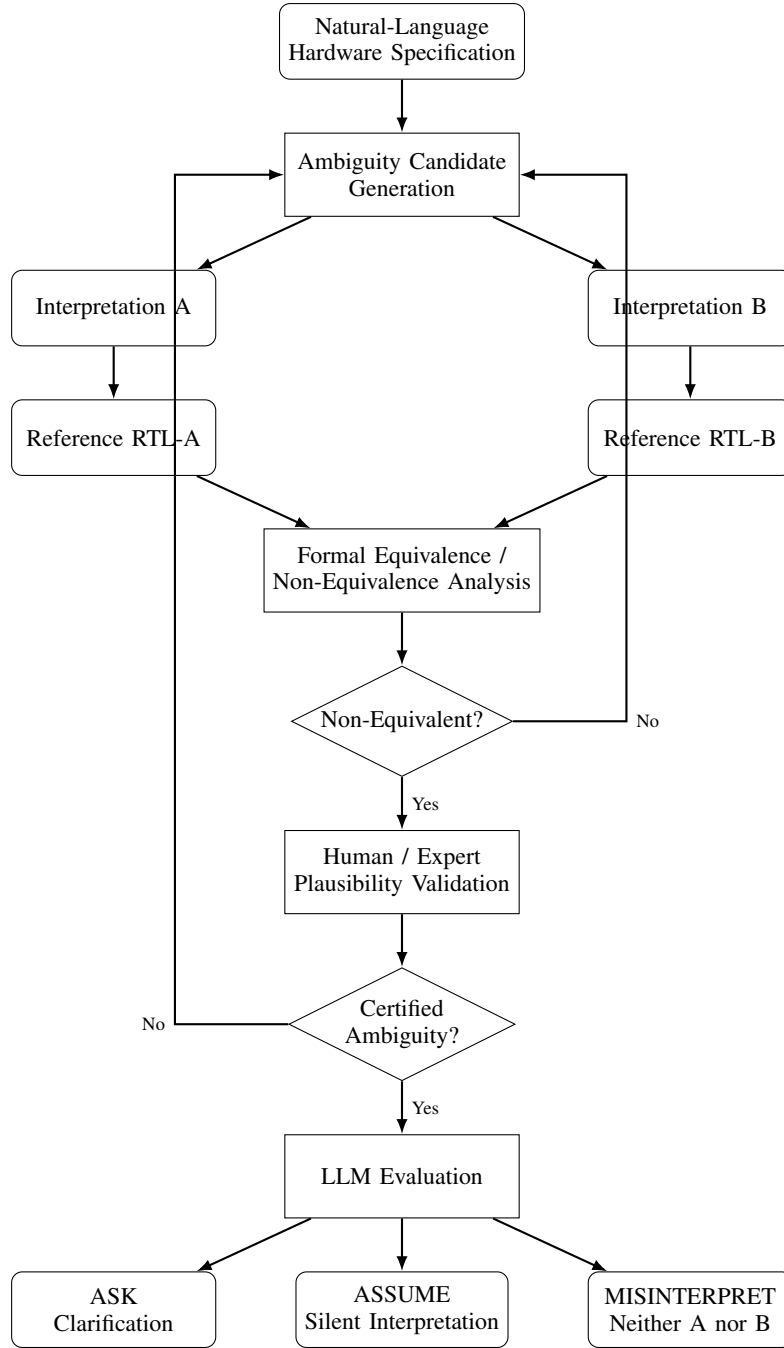


Fig. 1. Overview of the proposed FORMAL-AMBIG-RTL framework. Candidate ambiguities are converted into alternative interpretations and reference RTL implementations. Formal verification establishes behavioral distinction, while independent plausibility assessment determines whether both interpretations are reasonable. Certified cases are then used to evaluate LLM ambiguity-handling behavior.